LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK DAN QUEUE

Oleh:

Naila Atresia Rahma **NIM. 2510817120014**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL I

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Naila Atresia Rahma
NIM : 2510817120014

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 IMPLEMENTASI STACK 1
A. Source Code 1
B. Pembahasan 2
SOAL 2 SI PALUI DAN MESIN HITUNG ANEH 5
A. Source Code 6
B. Output Program 7
C. Pembahasan 8
SOAL 3 IMPLEMENTASI QUEUE 9
A. Source Code 9
B. Pembahasan 10
SOAL 4 SI PALUI DAN DERETAN ANGKA 9
A. Source Code 10
B. Output Program 11
C. Pembahasan 11
TAUTAN GIT 11

DAFTAR GAMBAR

Gambar 1. Screenshot hasil jawaban Soal 2A 7
Gambar 2. Screenshot hasil jawaban Soal 2B 7
Gambar 3. Screenshot hasil jawaban Soal 2C 7
Gambar 4. Screenshot hasil jawaban Soal 3A 11
Gambar 5. Screenshot hasil jawaban Soal 3B 11
Gambar 6. Screenshot hasil jawaban Soal 3C 11

DAFTAR TABEL

Tabel 1. Source Code stack.cpp..... 1
Tabel 2. Source Code prob1.cpp 6
Tabel 3. Source Code queue.cpp 9
Tabel 4. Source Code prob2.cpp 10

SOAL 1 IMPLEMENTASI STACK

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw. Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1. Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}


bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    if (isEmpty(s)) return false;
    return (s->top == &(s->data[MAX - 1]));
}
void push(Stack* s, int value) {
    if (isFull(s)) {
            throw  std::overflow_error("Stack  Overflow: Kapasitas penuh!");
    }

    if (isEmpty(s)) {
        s->top = &(s->data[0]);
    } else {
        s->top++;
    }

    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
```

```
 throw   std::underflow_error("Stack   Underflow:   Stack kosong!");
    }

    if (s->top == &(s->data[0])) {
        s->top = nullptr;
    } else {
        s->top--;
    }
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::runtime_error("Stack Empty: Tidak ada data!");
    }
    return *(s->top);
}
```

B. Pembahasan

Alur Algoritma :

- init: Menyetel pointer top ke nullptr untuk menandakan tumpukan kosong.
- push: Memeriksa apakah tumpukan penuh (isFull). Jika tidak, pointer top digeser ke alamat memori berikutnya (atau ke indeks 0 jika pertama kali) dan nilai baru dimasukkan ke alamat tersebut.
- pop: Memeriksa apakah tumpukan kosong (isEmpty). Jika tidak, pointer top digeser mundur ke elemen sebelumnya. Jika itu elemen terakhir, top dikembalikan ke nullptr.
- peek: Langsung mengembalikan nilai yang ditunjuk oleh pointer top tanpa mengubah posisi pointer.
- throw: Digunakan untuk menangani kondisi invalid seperti Stack Overflow (saat push pada kondisi penuh) atau Stack Underflow (saat pop pada kondisi kosong).

Justifikasi Kebenaran :

Algoritma ini benar karena menggunakan prinsip LIFO (Last-In, First-Out). Penggunaan pointer top memastikan bahwa operasi penambahan dan penghapusan selalu terjadi pada satu ujung yang sama, yaitu elemen paling atas.

Analisis Kompleksitas :

- `init, isEmpty, isFull`

  Time Complexity : O(1)

  Space Complexity : O(1)
- `push, pop, peek`

  Time Complexity : O(1)

  Space Complexity : O(1)

Pembahasan Code stack.cpp :

- Baris 1: Program akan manggil file `stack.h` yang sudah kita buat, supaya si program kenal sama struktur tumpukannya.
- Baris 2 : Pakai `stdexcept` buat jaga-jaga kalau nanti ada error, biar programnya bisa ngasih tahu kalau ada masalah (kayak tumpukan kepenuhan).
- Baris 4: Fungsi ini akan dipanggil pas kita pertama kali mau buat tumpukan baru
- Baris 5: Di sini, posisi `top` (puncak) tumpukannya disetel ke `nullptr` atau kosong, artinya tumpukannya masih benar benar kosong
- Baris 10: Baris ini cuma mau ngecek, kalau `top`-nya masih kosong, berarti tumpukan itu emang belum ada isinya.
- Baris 14-15: Fungsi ini akan ngecek apakah tumpukan sudah penuh. Caranya, dia liat apakah si `top` sudah menunjuk ke data yang paling terakhir (`MAX - 1)`. Kalau iya, berarti sudah tidak bisa nambah lagi nantinya.
- Baris 18-20: Sebelum masukin data, program akan cek dulu pakai `isFull`. Kalau kepenuhan, nanti akan "Stack Overflow!" dan prosesnya berhenti.
- Baris 22-23: Kalau ternyata tumpukannya masih kosong banget, si `top` akan diarahkan ke alamat data pertama (indeks 0).

- Baris 24-25: Tapi kalau sudah ada isinya, posisi `top`-nya akan naik satu tingkat ke atas buat nyiapin tempat baru.
- Baris 28: Nah, di sini nilai `value` yang akan benar-benar disimpan di posisi puncak yang sudah disiapin tadi.
- Baris 32-34: Program akan cek dulu, kalau tumpukannya kosong tapi dipaksa ambil data, nanti akan ngeluarin peringatan "Stack Underflow!".
- Baris 36-37: Kalau ternyata yang mau diambil itu adalah data terakhir yang tersisa, maka `top`-nya akan balik lagi jadi kosong (`nullptr`).
- Baris 38-39: Tapi kalau masih ada data lain di bawahnya, posisi `top` tinggal turun satu tingkat aja buat pindah ke data di bawahnya.
- Baris 44-46: Sama kayak tadi, program akan mastiin dulu kalau tumpukannya tidak kosong sebelum kita intip isinya.
- Baris 47: Kalau aman, program akan ngasih tahu kamu nilai apa sih yang ada di posisi paling atas tumpukan itu sekarang, tanpa menghapus datanya.

SOAL 2 SI PALUI DAN MESIN HITUNG ANEH

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

• sebuah bilangan bulat, atau

• sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

• Mesin membaca simbol dari kiri ke kanan.

• Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.

• Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n adalah jumlah simbol dalam ekspresi

• Ai adalah nilai bilangan pada input

• Simbol yang valid hanya:

‣ bilangan bulat ‣ operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang simbol

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

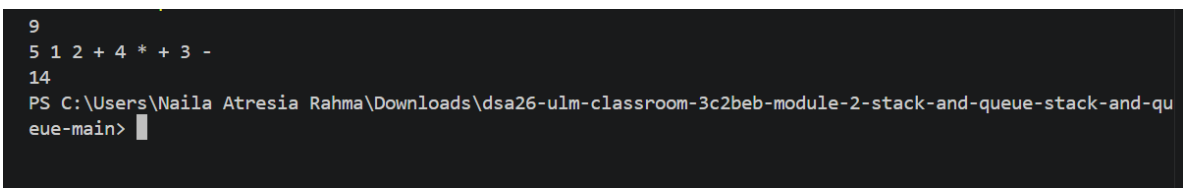
Tabel 2. Source Code prob1.cpp

```
 1 #include "stack.h"
 2 #include <iostream>
 3 #include <string>
 4
 5 using namespace std;
 6
 7 int main() {
 8     int n;
 9     if (!(cin >> n)) return 0;
10
11     Stack s;
12     init(&s);
13
14     string simbol;
15     for (int i = 0; i < n; i++) {
16         cin >> simbol;
17         if (simbol == "+" || simbol == "-" || simbol ==
   "*" || simbol == "/") {
18             int bil2 = peek(&s); pop(&s);
19             int bil1 = peek(&s); pop(&s);
20             int hasil;
```

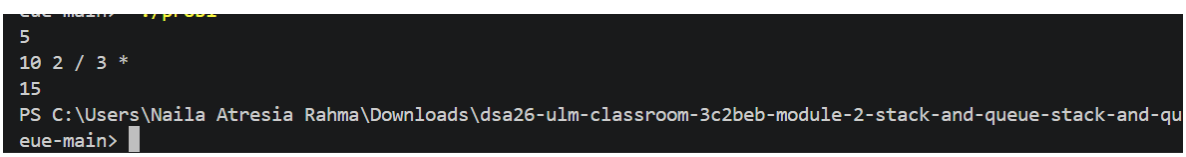
```
            if (simbol == "+") hasil = bil1 + bil2;
            else if (simbol == "-") hasil = bil1 - bil2;
            else if (simbol == "*") hasil = bil1 * bil2;
            else if (simbol == "/") {
                                if (bil2 == 0) throw runtime_error("Division by zero");
                hasil = bil1 / bil2;
            }
            push(&s, hasil);
        } else {
            int angka = stoi(simbol);
            push(&s, angka);
        }
    }
    cout << peek(&s) << endl;

    return 0;
}
```

B. Output Program

Gambar 1. Screenshot hasil jawaban Soal 2A



Gambar 2. Screenshot hasil jawaban Soal 2B



Gambar 3. Screenshot hasil jawaban Soal 2C

C. Pembahasan

Prob1.cpp:

- Baris 1-3: Program akan memanggil file `stack.h`, `iostream`, dan `string`. Ini dilakukan supaya alat-alat buat tumpukan data dan pengolah teksnya siap dipakai nanti.
- Baris 5: `using namespace std` dipakai supaya penulisan kodenya lebih ringkas dan tidak perlu ngetik `std::` terus-menerus.
- Baris 8-9: Di sini program akan buat variabel n untuk menentukan berapa banyak input yang masuk. Kalau inputnya tidak ada, program akan langsung berhenti.
- Baris 11-12: Objek stack s akan disiapkan dan fungsi `init` bakal dijalankan buat mastiin tumpukannya dalam keadaan kosong.
- Baris 15-16: Program akan buat perulangan sebanyak n kali. Setiap putaran, satu simbol (bisa angka atau operator) akan dibaca dari layar.
- Baris 17: Program akan cek apakah simbol itu adalah operator matematika seperti +, -, *, atau /.
- Baris 18-19: Kalau iya operator, program akan mengambil dua angka paling atas dari tumpukan lewat fungsi peek dan pop. Angka pertama yang keluar jadi bil2, dan angka kedua jadi bil1.
- Baris 21-26: Sesuai jenis operatornya, kedua angka tadi akan dihitung. Khusus pembagian (/), ada pengecekan supaya tidak membagi dengan angka nol agar tidak error nantinya.
- Baris 27: Hasil hitungannya akan dimasukkan lagi ke dalam tumpukan menggunakan fungsi `push`.
- Baris 29-31: Kalau simbolnya bukan operator (berarti angka), teks tersebut akan diubah jadi bilangan bulat pakai stoi, lalu angka itu akan langsung disimpan ke dalam tumpukan.
- Baris 34: Setelah semua input selesai diproses, hasil akhir hitungannya akan diambil dari puncak tumpukan dan ditampilkan ke layar.
- Baris 36: Program akan lapor kalau semua tugas sudah beres dan selesai bekerja.

SOAL 3 IMPLEMENTASI QUEUE

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw. Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3. Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;
    return (q->rear == &(q->data[MAX - 1]));
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
            throw  std::overflow_error("Queue  Overflow: Antrean penuh!");
    }

    if (isEmpty(q)) {
        q->front = &(q->data[0]);
        q->rear = &(q->data[0]);
    } else {

        q->rear++;
    }

```

```
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
            throw std::underflow_error("Queue Underflow: Antrean kosong!");
    }

    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {

        q->front++;
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) {
        throw std::runtime_error("Queue Empty: Tidak ada elemen di depan.");
    }
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw std::runtime_error("Queue Empty: Tidak ada elemen di belakang.");
    }
    return *(q->rear);
}
```

B. Pembahasan

Alur Algoritma :

- init: Menyetel pointer *front* dan *rear* ke `nullptr` untuk menandakan bahwa antrean masih dalam keadaan kosong.
- enqueue: Memeriksa apakah antrean sudah penuh (`isFull`). Jika masih tersedia ruang, program akan memasukkan nilai baru ke posisi *rear*. Jika antrean awalnya kosong, pointer *front* dan *rear* akan diarahkan ke elemen pertama tersebut; jika tidak, hanya pointer *rear* yang bergeser maju ke elemen baru.

- dequeue: Memeriksa apakah antrean kosong (`isEmpty`). Jika ada isinya, elemen pada posisi *front* akan dihapus atau dilewati. Jika setelah penghapusan antrean menjadi kosong, kedua pointer (*front* dan *rear*) dikembalikan ke `nullptr`.
- peek / front: Langsung mengembalikan nilai data yang berada di posisi *front* (paling depan) tanpa mengubah struktur atau posisi antrean.
- throw: Digunakan untuk menangani kondisi tidak valid seperti *Queue Overflow* (menambah data saat penuh) atau *Queue Underflow* (mengambil data saat kosong) agar program tidak *crash* nantinya.

Justifikasi Kebenaran :

Algoritma ini benar karena menerapkan prinsip FIFO (First-In, First-Out) secara konsisten. Penggunaan pointer *front* memastikan data yang masuk pertama kali akan dilayani lebih dulu, sementara pointer *rear* memastikan data baru selalu ditambahkan di akhir urutan, sehingga integritas antrean tetap terjaga.

Analisis Kompleksitas :

- `init, isEmpty, isFull`
  Time Complexity : O(1)
  Space Complexity : O(1)
- `enqueue, dequeue, peek`
  Time Complexity : O(1)
  Space Complexity : O(1)

Pembahasan Code queue.cpp :

- Baris 1: Program akan memanggil file `queue.h` supaya semua definisi struktur antrean yang sudah dibuat sebelumnya bisa dikenali.
- Baris 2: Di sini `stdexcept` dipanggil buat menyiapkan pesan error nantinya kalau ada masalah di antrean.
- Baris 4: Fungsi ini akan buat persiapan awal saat antrean pertama kali digunakan.
- Baris 5 - 6: Pointer `front` (depan) dan `rear` (belakang) langsung disetel ke `nullptr` untuk menandakan antreannya masih kosong melompong.

- Baris 10: Baris ini cuma mengecek, kalau `front`-nya masih kosong, berarti antreannya memang belum ada isinya.
- Baris 14: Sebelum bilang penuh, program akan cek dulu lewat `isEmpty`. Kalau kosong, ya tidak mungkin penuh.
- Baris 15: Di sini program akan melihat apakah posisi `rear` sudah sampai di alamat data paling terakhir `(MAX - 1)`. Kalau iya, berarti antreannya sudah mentok.
- Baris 19-21: Program akan peringatan "Queue Overflow!" kalau dipaksa nambah data padahal antreannya sudah penuh.
- Baris 23-25: Kalau antrean masih kosong, posisi `front` dan `rear` akan disatukan menunjuk ke data pertama di indeks 0.
- Baris 27-28: Tapi kalau sudah ada isinya, posisi `rear`-nya saja yang bergeser maju satu langkah buat nyiapin tempat.
- Baris 31: Nah, di baris ini nilai barunya benar-benar disimpan di posisi `rear` yang sudah disiapkan tadi.
- Baris 35-37: Kalau antrean kosong tapi mau diambil datanya, program akan ngasih peringatan "Queue Underflow!".
- Baris 39-42: Di sini program mengecek: kalau `front` dan `rear` ada di posisi yang sama (berarti tinggal satu data terakhir), maka setelah diambil, antreannya balik lagi jadi nullptr alias kosong.
- Baris 43-44: Kalau datanya masih banyak, posisi front tinggal digeser maju saja ke urutan berikutnya.
- Baris 49-51: Program akan cek dulu, jangan sampai ngintip antrean yang kosong.
- Baris 52: Kalau aman, nilai di posisi depan (*front*) akan dikasih tahu tanpa menghapus datanya.
- Baris 56-59: Sama seperti tadi, tapi yang ini buat ngintip siapa sih yang ada di urutan paling belakang (*rear*).

SOAL 4 SI PALUI DAN DERETAN ANGKA

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

•$1 \leq n \leq 100$

•$-1 \leq k \leq n$

•$-1000 \leq Ai \leq 1000$

Keterangan:

•n: jumlah elemen

•k: ukuran jendela

•Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

•Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk

•Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

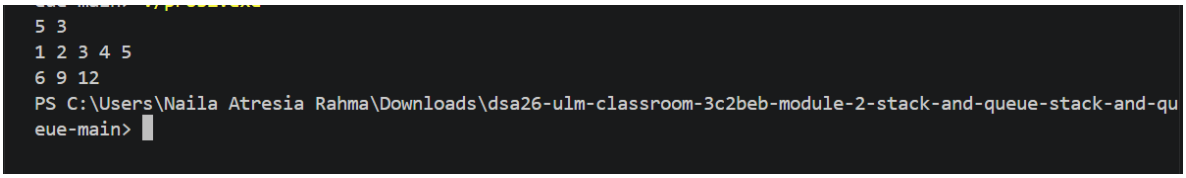
A. Source Code

Tabel 4. Source Code prob2.cpp

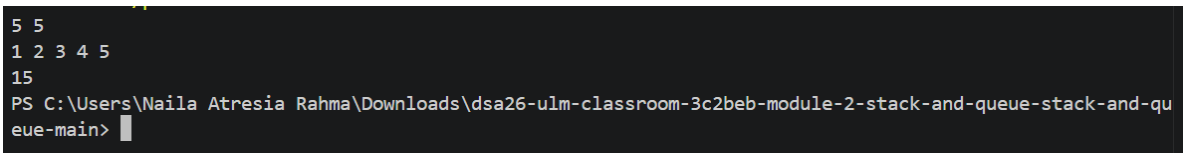
```
#include "queue.h"
#include <iostream>

using namespace std;

int main() {
    int n, k;
    if (!(cin >> n >> k)) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    if (k <= 0 || k > n) return 0;

    Queue q;
    init(&q);

    long long current_sum = 0;

    for (int i = 0; i < k; i++) {
        enqueue(&q, arr[i]);
        current_sum += arr[i];
    }
    cout << current_sum;

    for (int i = k; i < n; i++) {
        int keluar = front(&q);
        dequeue(&q);

```

```
        enqueue(&q, arr[i]);

        current_sum = current_sum - keluar + arr[i];

        cout << " " << current_sum;
    }
    cout << endl;
    return 0;
}
```

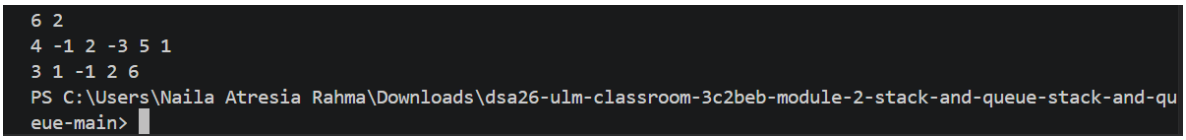
B. Output Program



Gambar 4. Screenshot hasil jawaban Soal 3A



Gambar 5. Screenshot hasil jawaban Soal 3B



Gambar 6. Screenshot hasil jawaban Soal 3C

C. Pembahasan

Prob2.cpp:

- Baris 1: `(#include "queue.h")` Di baris ini, program akan memanggil file header yang sudah dibuat sebelumnya. Hal ini perlu dilakukan supaya kode di file ini kenal sama struktur Queue dan fungsi-fungsi yang mau kita isi isinya nanti.
- Baris 2 : `(#include <stdexcept>)` Baris ini digunakan untuk memasukkan pustaka standar C++ yang menangani kesalahan (*exception*). Pustaka ini penting banget karena akan kita pakai buat fungsi throw saat antrean kepenuhan atau kosong biar program tidak error mendadak.

- Baris 4-7 (`init`): Fungsi ini akan mengatur nilai awal antrean dengan menyetel pointer `front` dan `rear` ke `nullptr`. Hal ini dilakukan untuk menandakan bahwa antrean dalam keadaan kosong saat pertama kali dibuat.
- Baris 9-11 (`isEmpty`): Program akan memeriksa apakah pointer `front` bernilai `nullptr`. Jika iya, maka fungsi akan mengembalikan nilai benar yang berarti antrean memang kosong.
- Baris 13-16 (`isFull`): Sistem akan mengecek apakah antrean sudah penuh dengan melihat apakah pointer `rear` sudah menunjuk ke alamat memori elemen terakhir dalam array `(MAX - 1)`.
- Baris 18-21: Sebelum menambah data, program akan memastikan status antrean lewat `isFull`. Jika penuh, sistem akan menghentikan proses dan memberikan pesan error "Queue Overflow" melalui mekanisme `throw`.
- Baris 23-26: Jika antrean masih kosong, program akan mengarahkan baik pointer `front` maupun `rear` ke alamat memori elemen pertama di indeks 0.
- Baris 27-29: Jika sudah ada isinya, hanya pointer `rear` yang akan digeser maju ke alamat memori berikutnya untuk menyiapkan tempat bagi data baru.
- Baris 31: Nilai yang dimasukkan (`value`) akan disimpan tepat di alamat memori yang ditunjuk oleh pointer `rear`.
- Baris 34-37: Program akan mengecek apakah antrean kosong lewat `isEmpty`. Jika tidak ada data, sistem akan memberikan pesan error "Queue Underflow".
- Baris 39-42: Di sini ada pengecekan khusus; jika pointer `front` dan `rear` menunjuk ke alamat yang sama (artinya hanya ada satu data tersisa), maka setelah data diambil, kedua pointer akan dikembalikan ke `nullptr`.
- Baris 43-45: Jika datanya lebih dari satu, program akan menggeser pointer `front` maju satu posisi ke elemen berikutnya.
- Baris 48-51: Fungsi `front` akan memastikan antrean tidak kosong sebelum memberikan informasi. Jika kosong, pesan error akan muncul.
- Baris 52: Program akan mengembalikan nilai yang berada di posisi paling depan tanpa menghapus elemen tersebut.

- Baris 55-60: Fungsi `back` bekerja dengan cara yang sama, namun nilai yang diberikan adalah data yang berada di posisi paling belakang atau yang paling terakhir dimasukkan.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-nailaarahma1912-dot